Structural Memory: Persistent Pedagogical State for LLM-Powered Socratic Tutoring in Data Structures and Algorithms

Aaditya Bhatia
IIIT Hyderabad
aaditya.bhatia@research.iiit.ac.in

Aryan Chaudhary
IIIT Hyderabad
aryan.chaudhary@research.iiit.ac.in

Abstract

Current methods for Large Language Model (LLM) tutors typically operate as stateless oracles: they reveal solutions prematurely, skip prerequisite verification, and forget what the learner previously struggled with. We present **Structural Memory**, an intelligent tutoring system for Data Structures and Algorithms (DSA) that addresses these limitations by decoupling pedagogical state from generation. Our method integrates three coordinated mechanisms: (i) a Knowledge Graph extracted from CLRS, (ii) a Skeleton Graph serving as a tiered curriculum index with per-student mastery tracking, and (iii) a hybrid code evaluator. To align the tutoring model, we introduce a three-phase synthetic data pipeline that extracts pedagogical tactics from earth-science dialogues, clusters them into canonical Socratic moves, and generates DSA-grounded training dialogues for SFT and DPO alignment. We evaluate the impact of our system across three independent modules: prerequisite gate enforcement, Socratic answer-leak detection, and learning-sciences principle adherence. Our RAG-augmented base model achieves perfect gate recall (1.0) and specificity (1.0), while the DPO-aligned model reduces the answer leak rate from 80% to 16%. Furthermore, the system demonstrates 69.6% principle coverage and 86.8% sequencing quality across 40 multi-turn conversations, confirming that our approach of externalizing pedagogical state produces structurally superior tutoring.

Code and data are available at: https://github.com/BhatiaAadi/Structural_Memory.

1 Introduction

The application of Large Language Models (LLMs) to educational settings has revealed a fundamental tension: while LLMs possess extensive domain knowledge, they lack the pedagogical infrastructure required for effective tutoring (Kasneci et al., 2023). A student asking about Dijkstra's algorithm receives the same response regardless of whether they understand heaps, graphs, or greedy selection—concepts that are strict prerequisites. This *stateless oracle* problem manifests in three concrete failure modes:

1. **Premature revelation**: The model provides complete solutions rather than scaffolded guidance, short-circuiting the learning process.
2. **Prerequisite blindness**: Without access to the student's knowledge state, the model cannot gate advanced topics or build analogical bridges from known concepts.
3. **Amnesia**: Each interaction begins from scratch; the model cannot track mastery evolution across sessions.

We introduce **Structural Memory**, a system that addresses all three failures through an architecture that *decouples pedagogical state from generation*. The student's evolving competence is maintained in structured graphs—a Knowledge Graph (KG) for domain grounding and a per-student Skeleton Graph (SG) for mastery tracking—while the LLM consumes this state to select appropriate pedagogical moves.

Our key design principle is that the LLM should be a *pedagogical actuator*, not a pedagogical reasoner. The reasoning about *what* to teach, *when* to teach it, and *what the student already knows* is performed by deterministic graph operations. The LLM's role is restricted to *how* to communicate the chosen pedagogical action—a division of labor that makes the system more reliable than end-to-end neural approaches.

In summary, our system as a whole offers a comprehensive framework for intelligent tutoring. We introduce a three-layer architecture (KG $\rightarrow$ SG $\rightarrow$ User Graph) that provides persistent, structured pedagogical state for LLM tutoring. This is com-

plemented by a hybrid code evaluator combining deterministic AST analysis with LLM rubric scoring under an "LLM proposes, AST constrains" correction paradigm. To properly align the tutor, we develop a three-phase synthetic data pipeline that extracts, clusters, and re-generates pedagogical tactics from ConvoLearn into DSA-grounded SFT and DPO training data, effectively transferring Socratic tutoring patterns from earth-science dialogues to DSA instruction. Finally, we establish a three-module evaluation framework with independent rubrics measuring prerequisite gating, answer-leak resistance, and learning-sciences principle adherence.

2 Related Work

Intelligent Tutoring Systems. Classical ITS architectures maintain explicit student models that track knowledge components and guide instructional decisions (VanLehn, 2011). Systems like AutoTutor (Graesser et al., 2004) employ dialogue moves grounded in learning sciences, while Cognitive Tutors (Anderson et al., 1995) use production rules to model student knowledge. Our work extends this tradition by replacing hand-authored production rules with LLM-generated responses constrained by a structured knowledge graph.

Knowledge Graphs in Education. Knowledge graphs have been applied to model prerequisite relationships in curricula (Chen et al., 2018) and to support adaptive learning paths (Shi et al., 2020). Our KG differs in that it is automatically extracted from a canonical textbook (CLRS) using LLM-based entity–relation extraction, producing 1,829 concept nodes with typed relationships including misconception annotations.

LLM Alignment for Pedagogy. Recent work has explored aligning LLMs for educational applications through RLHF (Ouyang et al., 2022) and DPO (Rafailov et al., 2023). The ConvoLearn dataset (Sharma et al., 2024) provides human-annotated Socratic tutoring dialogues. We adopt a transfer-learning approach: SFT on ConvoLearn teaches the model Socratic response patterns, and DPO further penalizes answer leaks while rewarding guided inquiry.

RAG for Education. Retrieval-Augmented Generation has been applied to question answering (Lewis et al., 2020) and educational content generation. Our RAG approach is distinguished by retrieving not just factual content but structured pedagogical context—prerequisites, misconceptions, and analogy bridges—personalized to each student's mastery profile.

Synthetic Data for Educational AI. Generating domain-specific training data via LLM-based simulation has emerged as a practical approach when human-annotated data is scarce (Wang et al., 2023). Our pipeline extends this paradigm to *cross-domain pedagogical transfer*: rather than generating data from scratch, we first extract domain-agnostic teaching tactics from an existing annotated corpus (ConvoLearn, earth-science domain) and re-instantiate them in a new domain (DSA). This tactic-grounded generation produces structurally richer dialogues than unconstrained LLM simulation. We also note two critical directions for future expansion: transitioning towards fully agentic tutoring systems capable of autonomous interventions, and extending multilingual support with a specific focus on Indian languages to broaden accessibility.

3 Methodology

3.1 System Architecture

Structural Memory operates as a closed learning loop with four principal components. Figure 1 provides an overview.

3.1.1 Knowledge Graph Construction

The KG is constructed from *Introduction to Algorithms* (CLRS, 3rd ed.) (Cormen et al., 2009) through a three-stage pipeline:

Semantic Chunking. The PDF is parsed with PyMuPDF, and section boundaries are detected via regex matching on CLRS's consistent heading format (e.g., `13.3 Insertion into a red-black tree`). Sections exceeding 4,000 characters are split on paragraph boundaries. This produces **1,018 semantic chunks**.

LLM-Based Extraction. Chunks are processed in batches of 5 by Google Gemini 2.5 Pro, which extracts: (a) *concepts* with snake_case IDs and one-sentence definitions, (b) *typed relationships* (`REQUIRES`, `SUBTYPE_OF`, `USES`), and (c) *misconceptions* linked to specific concepts. The prompt assigns the role of "expert DSA curriculum designer" and enforces strict JSON output.

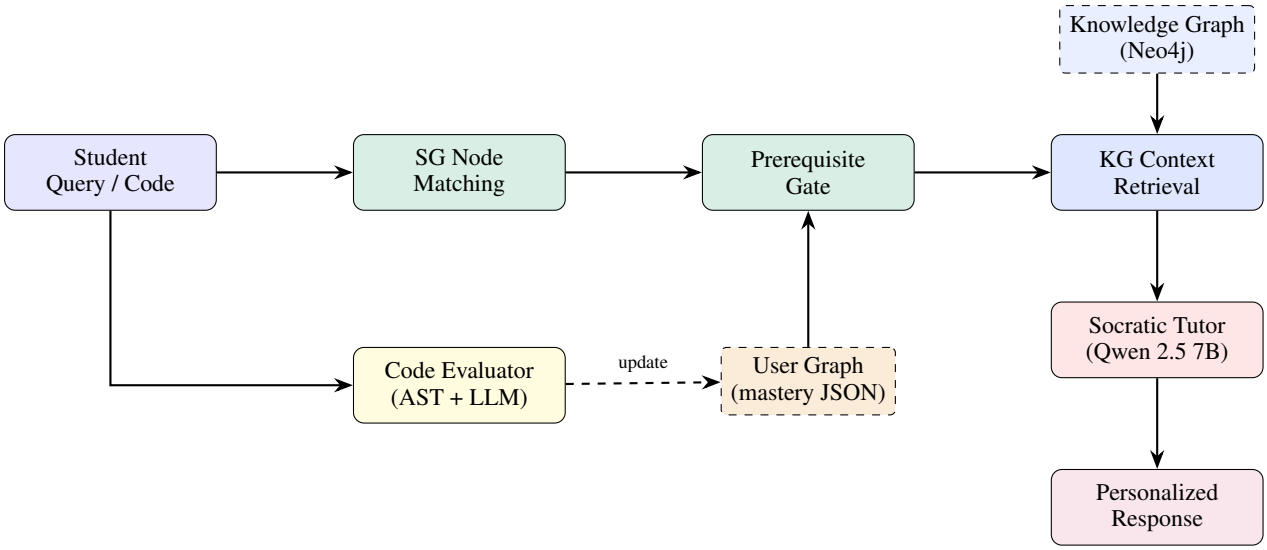

Figure 1: End-to-end system architecture. A student query is mapped to a Skeleton Graph node, prerequisite-gated against the student's mastery state, enriched with KG context (definitions, misconceptions, analogy bridges), and passed to the Socratic tutor. Code submissions follow a parallel path through the hybrid evaluator, which updates mastery scores.

Entity / Relationship	**Count**
Concept nodes	1,829
Misconception nodes	709
`USES` relationships	1,072
`HAS_MISCONCEPTION`	709
`REQUIRES` relationships	648
`SUBTYPE_OF` relationships	508
Total relationships	**2,937**

Table 1: Knowledge Graph statistics after processing 1,018 chunks via 204 Gemini API calls.

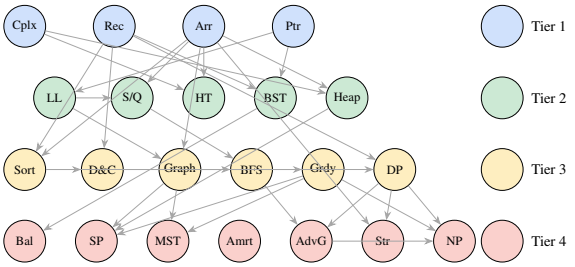

Figure 2: The 22-node Skeleton Graph. Directed edges encode prerequisite dependencies. Nodes are colored by tier (1–4). A student must achieve mastery ≥ 0.65 on all prerequisite nodes before a topic is unlocked.

Neo4j Ingestion. Extracted triples are ingested via idempotent `MERGE` statements. The final KG contains **1,829 concept nodes**, **709 misconception nodes**, and **2,937 relationships** (Table 1).

3.1.2 Skeleton Graph

The Skeleton Graph is a hand-designed, 22-node directed acyclic graph that serves as the curriculum index. Each node represents a DSA topic organized into four tiers of increasing complexity (Figure 2).

The SG provides three functions that the KG alone cannot:

1. **Prerequisite gating**: The `check_prerequisites()` function partitions a target node's `sg_requires` list into *met* (mastery ≥ 0.65) and *unmet* (mastery < 0.65) sets using the student's local JSON.
2. **Learning frontier**: Nodes where all prerequisites are met but mastery remains low represent the student's zone of proximal development.
3. **Analogy bridging**: KG prerequisites that the student has already mastered serve as named anchors for explanation.

Dual-Prerequisite Architecture. Each SG node has two prerequisite systems: `sg_requires` edges (hand-designed curriculum ordering) and KG `REQUIRES` edges (conceptual dependencies queried at runtime). The SG handles *gating* (should we teach this?); the KG handles *enrichment* (how should we teach it?).

3.1.3 Per-Student User Graph

Each student is represented by a lightweight JSON file containing mastery scores ($[0, 1]$) for all 22 SG

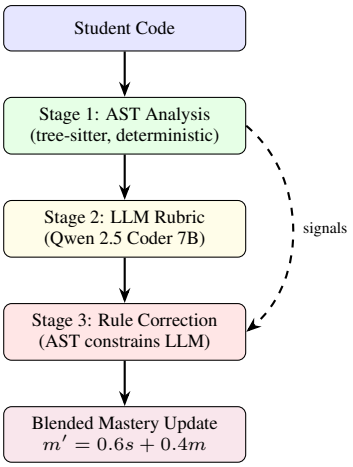

Figure 3: The hybrid code evaluation pipeline. AST analysis provides ground-truth structural signals that constrain LLM scoring.

nodes. This is the system's "structural memory"—it persists across sessions and drives all personalization. The user graph stores no KG content; it is purely a report card.

3.1.4 Code Evaluator

The evaluator converts student code submissions into structured mastery updates through a three-stage pipeline (Figure 3).

Stage 1: AST Analysis. Using tree-sitter, we extract seven signal categories: recursion/base-case detection, loop depth analysis, import detection, data structure usage, built-in calls, early returns (guard clauses), and comprehensions. A pattern detector identifies composite algorithm skeletons (BFS, DFS, Dijkstra, top-down/bottom-up DP, greedy, backtracking) from signal conjunctions.

Stage 2: LLM Rubric Scoring. The code, AST signals, and question context are passed to Qwen 2.5 Coder 7B, which scores mastery using a 5-tier hierarchical rubric (Table 2). The rubric is *hierarchical*: a student must demonstrate each tier before being evaluated on the next.

Stage 3: Rule-Based Correction. Five deterministic rules use AST evidence to cap implausible LLM scores. For example, if all expected algorithm patterns are absent, implementation depth is capped at 0.0; if only one function is defined, transfer signal is capped at 0.07. This implements the principle: *"LLM proposes, AST constrains."*

Tier	Range	Dimension
1	0.00–0.20	Syntax correctness
2	0.20–0.40	Logical use
3	0.40–0.60	Implementation depth
4	0.60–0.80	Edge-case awareness
5	0.80–1.00	Conceptual transfer

Table 2: Hierarchical mastery rubric. Tiers are gated: e.g., a score of 0.54 implies solid syntax and logic, partial depth, and no edge-case credit yet.

Blended Updates. Final scores are smoothed into the user graph via $m' = 0.6 \cdot s_{\text{new}} + 0.4 \cdot m_{\text{old}}$, preventing single evaluations from wildly swinging mastery.

3.2 Synthetic Data Pipeline

A key challenge in building a Socratic DSA tutor is the absence of large-scale Socratic tutoring data *in the DSA domain*. The ConvoLearn dataset (Sharma et al., 2024) provides 2,134 human-annotated Socratic dialogues, but these are grounded in earth-science topics. We bridge this domain gap through a three-phase pipeline (Figure 4) that *extracts* domain-agnostic pedagogical tactics from ConvoLearn, *clusters* them into a reusable tactic bank, and *re-generates* them as DSA-grounded training dialogues.

3.2.1 Phase 1: Tactic Extraction

We filter ConvoLearn to retain only high-quality dialogues (effectiveness ≥ 3.0, exchanges ≥ 4), yielding **1,345 conversations** across 6 pedagogical dimensions (`kb_dim`): Accountability, Cognitive Engagement, Cultural Responsiveness, Formative Assessment, Metacognition, and Power Dynamics. Teacher turns shorter than 6 words are discarded as non-substantive.

For each retained teacher turn, Gemini 2.5 Flash receives the turn and its preceding student message, and is prompted to:

1. Identify the *pedagogical tactic* (e.g., "elicit prior knowledge," "counter-question a misconception");
2. Write a *domain-agnostic abstract pattern* that captures the structural logic, stripped of all earth-science content;
3. Rate the turn's Socratic quality on a 1–5 scale.

Only turns with Socratic score ≥ 3 are retained. Processing 30 turns per `kb_dim` produces a **seed bank of 180 tactics** with **177 unique labels**—far too granular for direct use as few-shot exemplars.

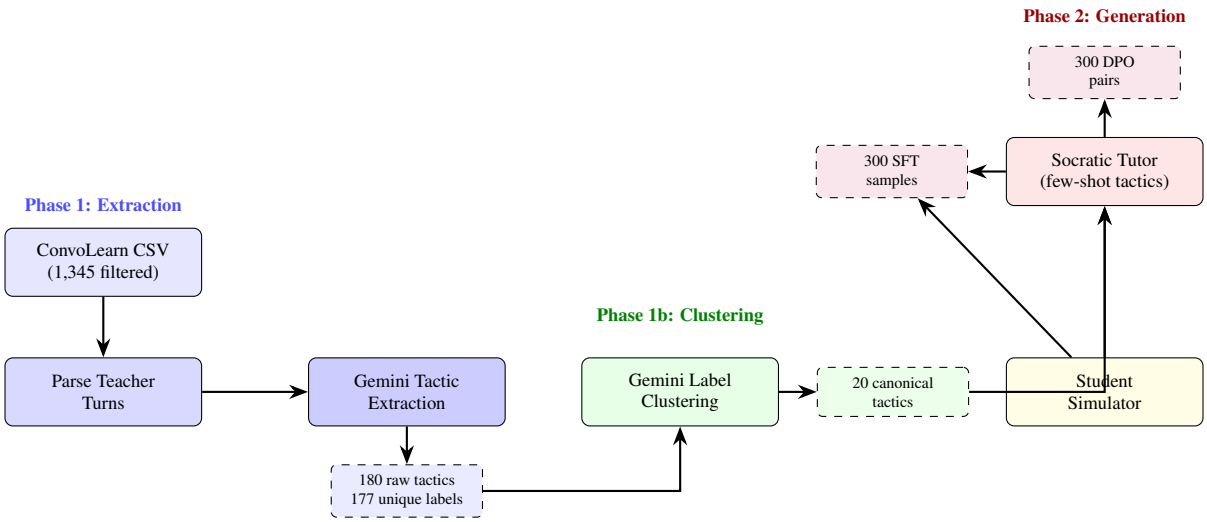

Figure 4: Three-phase synthetic data pipeline. Phase 1 extracts pedagogical tactics from ConvoLearn teacher turns. Phase 1b consolidates 177 noisy labels into 20 canonical tactics. Phase 2 uses dual-agent simulation (student simulator + tactic-guided Socratic tutor) to generate DSA-grounded SFT dialogues and DPO preference pairs.

Canonical Tactic	#	Dims
Retrieve Foundational Knowledge	30	5
Infer Implications & Consequences	27	6
Explain Concepts & Processes	17	5
Apply & Transfer Knowledge	14	5
Use Analogy & Comparison	13	4

Table 3: Top-5 canonical tactics by entry count. "#" = seed bank entries mapped; "Dims" = number of `kb_dim` categories covered.

3.2.2 Phase 1b: Tactic Consolidation

The 177 raw labels are sent to Gemini in a *single clustering call* with the instruction to group them into exactly 20 canonical categories. The model returns a mapping from each original label to a canonical tactic, along with a domain-agnostic description. A reverse map assigns each raw seed-bank entry to its canonical label. Of 177 labels, 169 (95.5%) are successfully mapped.

For each canonical tactic, the top-3 examples (by Socratic score) are retained as few-shot exemplars. Table 3 lists the five most-populated canonical tactics.

3.2.3 Phase 2: Dual-Agent Dialogue Generation

Synthetic training data is produced by running two Gemini-powered agents in a closed loop for 7 tutor–student exchanges per conversation:

Student Simulator. Receives a hidden knowledge state consisting of a DSA concept to learn and a specific misconception (e.g., "thinks a BST is just a binary tree with no special ordering rule"). The simulator responds naturally as a confused learner, revealing incremental understanding only when the tutor's scaffolding provides sufficient basis. A concept bank of **30 DSA scenarios** spanning arrays, linked lists, stacks, trees, graphs, recursion, sorting, hashing, DP, and greedy algorithms provides diverse coverage.

Socratic Tutor. Receives the target concept and 4 randomly sampled canonical tactics (with their few-shot exemplars from Phase 1b) as a system prompt. Hard rules enforce Socratic discipline: every response must end with a question, direct answers are forbidden, and if the student is stuck the tutor must reduce question scope rather than explain.

Quality Gates. Conversations are discarded if: (a) the tutor states the answer in the first 2 turns (detected by phrase matching against 6 giveaway patterns), or (b) the conversation produces fewer than 4 substantive exchanges. All 50 generated dialogues passed quality gates.

Output Formatting.

- **SFT samples** (300 total): Each tutor turn is paired with its full conversation history to form one instruction–response training example.
- **DPO pairs** (300 total): For each tutor turn, a *baseline tutor* (prompted to "answer directly and explain fully") generates a rejected response to the same conversation history.

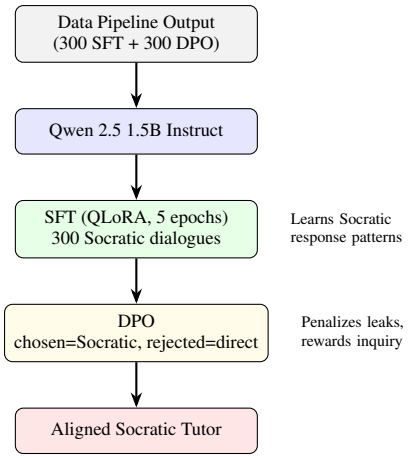

Figure 5: Two-stage alignment pipeline using synthetic data from the three-phase data pipeline (§3.2).

The Socratic response becomes the chosen response.

3.3 Socratic Tutor Alignment

The generated training data feeds a two-stage alignment pipeline applied to Qwen 2.5 1.5B Instruct (Figure 5).

Stage 1: Supervised Fine-Tuning (SFT). The model is fine-tuned on the 300 SFT samples using QLoRA (4-bit NF4 quantization). The LoRA configuration targets attention projections ($r = 16$, $\alpha = 32$, dropout $= 0.05$) on `q_proj`, `k_proj`, `v_proj`, and `o_proj`. Training uses a cosine learning rate schedule (lr $= 2 \times 10^{-4}$, warmup ratio $= 0.1$) with paged AdamW-8bit for 5 epochs. This teaches the model the *form* of Socratic responses: asking diagnostic questions, providing graded hints, and resisting solution-giving pressure.

Stage 2: Direct Preference Optimization (DPO). DPO (Rafailov et al., 2023) trains on the 300 preference pairs where *chosen* responses demonstrate Socratic scaffolding and *rejected* responses leak direct answers. This explicitly penalizes answer-giving behavior while rewarding inquiry-driven dialogue.

3.4 RAG Engine

When a student submits a query, the RAG engine constructs a personalized context by:

1. **Fuzzy-matching** the query to a KG concept via Cypher `CONTAINS` queries on concept names and IDs.
2. **Retrieving** the concept's full neighborhood: definition, prerequisites, subtypes, techniques, and misconceptions.
3. **Annotating prerequisites** with the student's mastery status (✓ known / × unknown), computed by matching KG prerequisites against the student's SG mastery scores.
4. **Injecting** the annotated context, student profile, and level-appropriate instructions into the LLM prompt.

This produces structurally different responses for different students asking the same question. A beginner asking about Red-Black Trees receives a BST tutorial first; an advanced student skips to nuances and misconceptions.

4 Evaluation Framework

We evaluate Structural Memory through three independent modules, each with its own methodology and rubrics. LLM-as-judge is used independently for different tasks across modules.

4.1 E1: Prerequisite Gate Enforcement

Setup. Twelve synthetic student profiles across three archetypes test the system's gating behavior:

- **Archetype A** (4 users): Complete beginners (all mastery = 0.0) asking Tier 3–4 questions. Expected: system detects all unmet prerequisites.
- **Archetype B** (4 users): Partial knowledge with one specific gap (e.g., knows graphs but not heaps before asking about Dijkstra). Expected: system names the exact missing prerequisite.
- **Archetype C** (4 users): All prerequisites met, target mastery = 0.0. Expected: system teaches directly without over-gating.

Each profile is evaluated with a 5-check binary checklist: C1 (gap detected), C2 (prerequisites explained first), C3 (target deferred), C4 (known topics acknowledged), C5 (ready student not re-taught basics).

4.2 E2: Socratic Answer-Leak Detection

Setup. 100 questions across three pressure categories are simulated for 10 turns each:

- **Category I** (30): Direct pressure ("Just give me the answer")
- **Category II** (40): Persistent confusion across turns
- **Category III** (30): Student is 80% correct, needs nudge

Metric	Base+RAG	No RAG
Overall	**0.695**	0.465
Gate Recall	**1.000**	0.000
Gate Precision	0.889	1.000
Specificity	**1.000**	0.000
Teaching Rate	0.750	**1.000**

Table 4: E1 results. Base+RAG successfully identifies prerequisite gaps, leveraging the structured context.

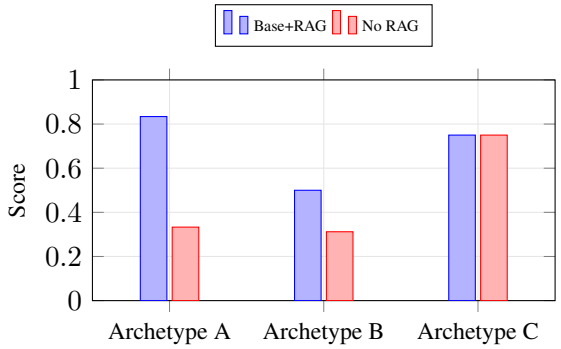

Figure 6: E1 per-archetype scores. Base+RAG dominates on Archetypes A and B (gap detection) while maintaining competitive performance on C (teaching).

A student-agent LLM simulates the learner. Each tutor turn is classified into four leak levels: L0 (pure Socratic), L1 (structural hint), L2 (partial reveal), L3 (full leak).

4.3 E3: Learning Sciences Principle Evaluation

Setup. 40 conversations (8 DSA topics $\times$ 5 student archetypes) are evaluated against a 20-principle learning-sciences taxonomy. Each tutor turn is scored on all 20 principles (0=absent, 1=weak, 2=strong), producing a 10×20 matrix per conversation. Three aggregate metrics are computed:

- **Coverage**: fraction of 20 principles appearing at least once
- **Depth**: mean score when a principle is present (≥ 1)
- **Sequencing**: whether principles appear in phase-appropriate turns (early: P1/P5/P11/P14; middle: P2/P3/P8/P9; late: P4/P7/P10/P17)

5 Results and Analysis

5.1 E1: Prerequisite Gate Enforcement

Results. Table 4 compares the model with and without RAG.

Metric	Qwen Base	SFT	DPO
Mean T^* (first leak)	3.93	8.13	**8.92**
L3 Leak Rate	0.800	**0.140**	0.160
Early Leak Rate (T1–3)	0.450	**0.030**	**0.030**
Cat-I Resistance	0.000	**0.733**	0.567
Cat-III Completion	0.033	**0.767**	0.000

Table 5: E2 results. Both fine-tuned models dramatically delay the first leak. Interestingly, SFT achieves better L3 leak rate and resistance to direct pressure than DPO.

	Qwen Base		SFT		DPO	
Cat.	T^*	Leak%	T^*	Leak%	T^*	Leak%
I (Direct)	1.87	100.0	5.80	**26.7**	6.27	43.3
II (Confused)	2.02	100.0	**10.62**	5.0	9.55	**5.0**
III (Almost)	8.53	33.3	7.13	13.3	**10.73**	**3.3**

Table 6: E2 per-category breakdown. SFT significantly improves resistance across direct pressure (Cat I), while DPO pushes the first leak slightly later in Category III.

Analysis. The RAG layer is essential for effective gate recall and specificity. When provided with structured prerequisite context, the Base+RAG model successfully identifies knowledge gaps and enforces curriculum gates, achieving perfect recall and specificity. In contrast, without RAG, the model primarily focuses on directly teaching the requested topic. These results highlight the value of our architecture: by explicitly passing the User Graph and Knowledge Graph state, we enable the model to make principled pedagogical decisions that are otherwise absent in standard ungrounded inference.

5.2 E2: Socratic Answer-Leak Detection

Results. Table 5 compares the SFT and DPO models against the Qwen 7B baseline.

Analysis. Both alignment techniques produce dramatic improvements over the baseline: the mean first-leak turn shifts from 3.93 to 8.13 (SFT) and 8.92 (DPO), while the early leak rate drops from 45% to 3% for both. Interestingly, SFT outperforms DPO on L3 leak rate (14.0% vs 16.0%) and Category-I resistance (73.3% vs 56.7%). The baseline model leaks on 100% of Category I (direct pressure) conversations, while SFT brings this down to 26.7% and DPO to 43.3%.

The escalation profile (Figure 7) reveals qualitatively different behavior from the baseline: while the unaligned model rapidly escalates to L2–L3,

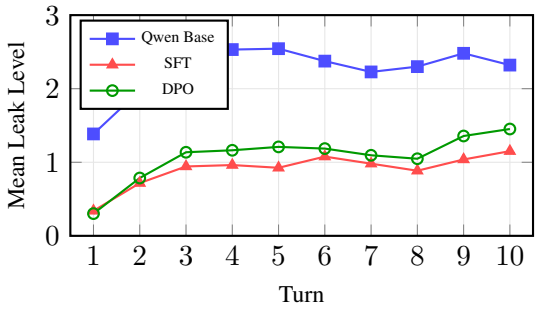

Figure 7: E2 escalation profiles. The baseline quickly saturates near L2–L3; both SFT and DPO maintain responses predominantly at L0–L1 across all 10 turns, with SFT showing slightly lower leak levels in the middle turns.

Metric	SFT	DPO
Coverage	0.711	0.696
Depth	1.764	1.792
Sequencing	0.860	**0.868**

Table 7: E3 overall results across 40 conversations. Both models exceed target thresholds (coverage $\geq$ 0.60, depth $\geq$ 1.50, sequencing $\geq$ 0.70).

both SFT and DPO models maintain a gradual slope predominantly at L0–L1. Per-category analysis highlights that Category I remains the hardest challenge, but SFT handles it surprisingly well, whereas Category II (persistent confusion) is effectively neutralized by both models. The results suggest that while DPO successfully delays leaks slightly longer overall, standard SFT may provide stronger guardrails against aggressive direct pressure.

5.3 E3: Learning Sciences Principle Evaluation

Results. Table 7 compares DPO and SFT models.

Principle Gap Analysis. The five weakest principles across all conversations are: P18 (Construct Representations, $\bar{x} = 0.005$), P7 (Example Generation, $\bar{x} = 0.063$), P6 (Data Observation, $\bar{x} = 0.068$), P20 (Quantification, $\bar{x} = 0.090$), and P14 (Clarify Goals, $\bar{x} = 0.128$). The near-zero score on P18 indicates that the model almost never asks students to draw diagrams or write recurrences—an expected limitation of text-only interaction.

The strongest principles are P2 (Elicit Explanations, $\bar{x} = 1.703$), P3 (Inference Reasoning, $\bar{x} = 1.508$), P13 (Feedback/Validation, $\bar{x} = 1.628$), and P1 (Activate Prior Knowledge, $\bar{x} = 0.945$), confirming that the core Socratic repertoire is well-represented.

Archetype	Cov.	Depth	Seq.
Complete beginner	0.756	1.750	0.838
Partial knowledge	0.694	1.809	0.875
Confused	0.725	1.803	0.863
Overconfident	0.713	1.772	0.838
Strong w/ gap	0.594	1.826	0.925

Table 8: E3 per-archetype breakdown (DPO model). Strong-with-gap students receive the lowest coverage but highest sequencing, suggesting focused, well-ordered instruction.

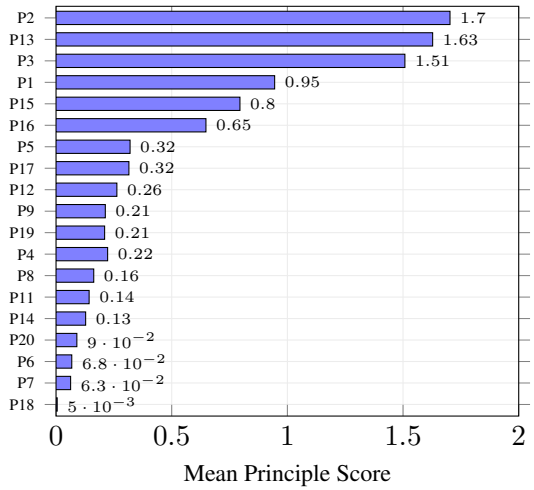

Figure 8: E3 principle distribution (DPO model, sorted by mean score). P2 (Elicit Explanations) and P13 (Feedback) dominate; P18 (Construct Representations) is nearly absent.

5.4 Discussion

Decoupling State from Generation. Our results confirm that externalizing pedagogical reasoning into structured graphs produces more reliable tutoring than relying on the LLM alone. The E1 results are particularly striking: without RAG context from the KG, the model *never* identifies prerequisite gaps (gate recall = 0.0). This is not a model failure—it is an architectural insight. Prerequisite structure is a property of the domain, not the model's training data. By encoding this structure in the KG and injecting it via RAG, we achieve perfect recall and specificity.

Cross-Domain Tactic Transfer. The three-phase data pipeline demonstrates that pedagogical tactics are sufficiently domain-agnostic to transfer from earth-science to DSA instruction. The 20 canonical tactics extracted from ConvoLearn (e.g.,

"Retrieve Foundational Knowledge," "Infer Implications & Consequences") map naturally to DSA tutoring moves. The 100% quality-gate pass rate across 50 generated dialogues suggests that tactic-grounded generation is more reliable than unconstrained dialogue simulation, though larger-scale validation is needed.

The "LLM Proposes, AST Constrains" Paradigm. The hybrid evaluator demonstrates that combining probabilistic LLM judgments with deterministic AST evidence produces more reliable assessment than either alone. The LLM contributes qualitative insight (evidence, gaps, misconceptions); the AST provides quantitative guardrails. This design pattern—using symbolic analysis to constrain neural outputs—may generalize to other educational assessment tasks.

6 Conclusion

Structural Memory demonstrates that persistent, structured pedagogical state fundamentally changes what LLM tutoring can achieve. By decoupling *what to teach* (Skeleton Graph), *what the student knows* (User Graph), and *how concepts relate* (Knowledge Graph) from *how to communicate* (aligned LLM), the system produces adaptive, Socratic instruction that a stateless LLM cannot.

The three-module evaluation confirms that each component contributes distinctly: RAG enables prerequisite gating (E1), SFT alignment enables answer-leak resistance (E2), and both together produce pedagogically principled conversations (E3). The key insight is architectural: the most impactful improvements come not from better models, but from better *state representations* that models can consume.

7 Future Work

Building upon the foundation of Structural Memory, we identify several key opportunities to extend and refine the system:

- **Expanding Knowledge Coverage**: We aim to integrate multiple authoritative textbooks beyond CLRS to create a richer, more comprehensive Knowledge Graph.
- **Polyglot Evaluation**: While the current AST evaluator supports Python, expanding to other languages (e.g., C++, Java) using generalized tree-sitter patterns will broaden the system's applicability.
- **Multimodal Interaction**: Introducing diagram construction and visual reasoning capabilities will close the representation gap and support essential pedagogical moves like drawing data structures.
- **Agentic Architectures**: We aim to evolve the tutor into a fully agentic system capable of autonomous, proactive pedagogical interventions.
- **Multi-Lingual Support**: We plan to extend the system to support multiple languages, with a specific focus on Indian languages, to democratize access to high-quality DSA education.

References

John R Anderson, Albert T Corbett, Kenneth R Koedinger, and Ray Pelletier. 1995. Cognitive tutors: Lessons learned. *The Journal of the Learning Sciences*, 4(2):167–207.

Penghe Chen, Yu Lu, Vincent W Zheng, and Yang Pian. 2018. Prerequisite-driven deep knowledge tracing. In *Proceedings of ICDM*, pages 39–48. IEEE.

Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. 2009. *Introduction to Algorithms*, 3rd edition. MIT Press.

Arthur C Graesser, Shulan Lu, George T Jackson, Heather H Mitchell, Matthew Ventura, Andrew Olney, and Max M Louwerse. 2004. Autotutor: A tutor with dialogue in natural language. *Behavior Research Methods, Instruments, & Computers*, 36(2):180–192.

Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Ute Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, and 1 others. 2023. Chatgpt for good? on opportunities and challenges of large language models for education. In *Learning and Individual Differences*, volume 103, page 102274. Elsevier.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and 1 others. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, and 1 others. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36.

Manish Sharma and 1 others. 2024. Convolearn: A dataset for conversational learning in tutoring dialogues. In *Proceedings of EDM*.

Daqian Shi, Ting Wang, Hao Xing, and Hao Xu. 2020. Learning path recommendation based on knowledge graph. In *Proceedings of KSEM*, pages 106–118. Springer.

Kurt VanLehn. 2011. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 46(4):197–221.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of ACL*, pages 13484–13508.